



Instituto **Tecnológico**[®]
de Aguascalientes



INSTITUTO TECNOLÓGICO DE AGUASCALIENTES

Integrante:

Manuel Humberto Reyes Salas

María José Guerrero Ríos

Jesús Eli Sánchez Ruvalcaba

Actividad:

Materia: Tecnologías inalámbricas

Docente: RICARDO ALEJANDRO RODRIGUEZ
JIMENEZ

Fecha: 26 de Mayo del 2026

Objetivo del Proyecto Previo

Establecer un canal de comunicación e infraestructura backend para que la ESP32 envíe datos de forma inalámbrica a través de una red WiFi. La información pasará por una API intermedia en **NodeJS** y se almacenará en una base de datos **SQL Server (MSSQL)** que se ejecuta de forma aislada dentro de un contenedor de **Docker**.

Arquitectura del Sistema

El sistema estará compuesto por los siguientes elementos:

- **ESP32:** Dispositivo encargado de generar y enviar los datos mediante WiFi.
- **Red WiFi:** Medio de comunicación entre la ESP32 y el servidor.
- **API NodeJS:** Servicio intermedio que recibe los datos desde la ESP32.
- **Docker:** Plataforma para la ejecución del contenedor del servidor de base de datos.
- **SQL Server (MSSQL):** Base de datos donde se almacenará la información.

Funcionamiento General del Sistema

El flujo de funcionamiento será el siguiente:

- La ESP32 se conecta a una red WiFi disponible.
- La ESP32 genera datos iniciales.
- Los datos son enviados mediante una petición HTTP hacia la API desarrollada en NodeJS.
- La API recibe los datos y los procesa.
- La API inserta la información en la base de datos SQL Server dentro del contenedor Docker.

Implementación del Sistema

Previo a la ejecución del proyecto final, se deberán realizar las siguientes configuraciones:

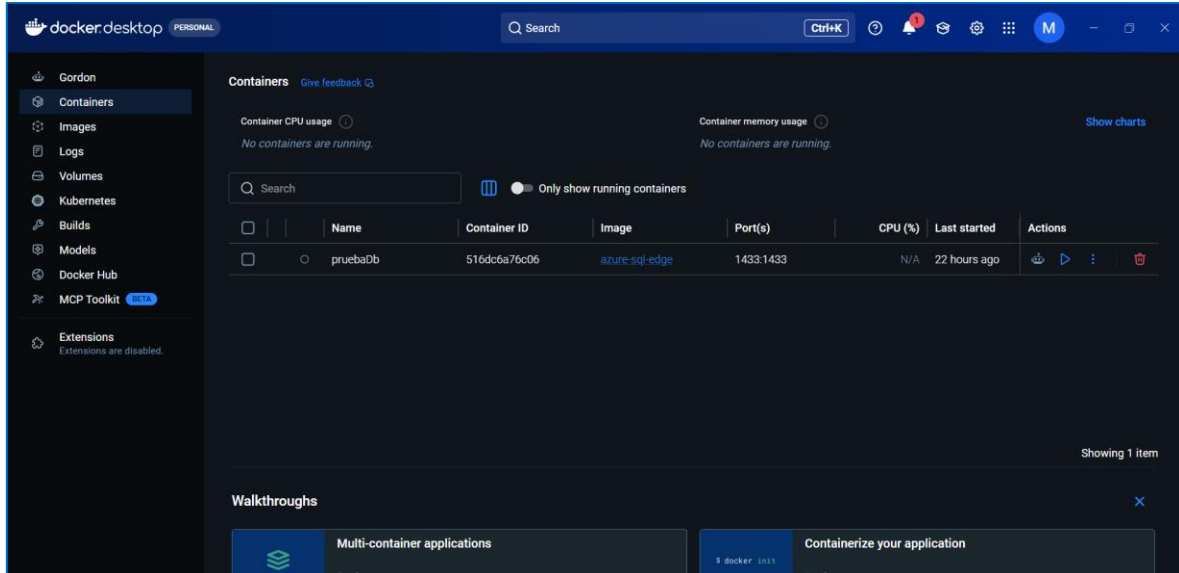
- Programar la ESP32 para conectarse a una red WiFi.
- Implementar el envío de datos mediante solicitudes HTTP.
- Desarrollar la API en NodeJS con un endpoint funcional para recibir datos.

- Configurar Docker para la ejecución del contenedor de SQL Server.
- Crear la base de datos y tablas necesarias para el almacenamiento de información.

Capturas del Código:

Captura del Docker: En esta parte creamos el contenedor en el docker , ejecutamos este código en powershell pero debemos de tener en cuenta el password y el usuario pues ya que después los utilizaremos.

```
PS C:\Users\mr411> docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=CONTR453N14" -p 1433:1433 --name pruebaDb --hostname pruebaDB -d mcr.microsoft.com/azure-sql-edge
Unable to find image 'mcr.microsoft.com/azure-sql-edge:latest' locally
latest: Pulling from azure-sql-edge
57f13386bc1b: Pull complete
f0412dfb1aae: Pull complete
dcc7b800eac9: Pull complete
c62c3fc0300d: Pull complete
Digest: sha256:902628a8be89e35dfb7895ca31d602974c7bafde4d583a0d0873844feb1c42cf
Status: Downloaded newer image for mcr.microsoft.com/azure-sql-edge:latest
516dc6a76c06ad870f30366a6949f2a16d220c256941282d5aabd899ccac0b59
PS C:\Users\mr411>
```



Capturas del PowerShell:

En esta parte estamos ubicándonos donde se encuentra nuestro proyecto para después iniciar el documento “servidor_api_esp32.js”

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\WINDOWS\System32> cd "C:\Users\mr411\OneDrive\Documentos\esp32_mssql_Definitivo-main"
PS C:\Users\mr411\OneDrive\Documentos\esp32_mssql_Definitivo-main> ls

Directorio: C:\Users\mr411\OneDrive\Documentos\esp32_mssql_Definitivo-main

Mode                LastWriteTime         Length Name
----                -
d-----          27/05/2026  01:34 a. m.             esp32_mssql_Definitivo
-a-----          27/05/2026  01:27 a. m.          113874 1.png
-a-----          27/05/2026  01:27 a. m.           83905 2.png
-a-----          27/05/2026  01:27 a. m.           90083 3.png
-a-----          27/05/2026  01:27 a. m.          61478 package-lock.json
-a-----          27/05/2026  01:27 a. m.           358 package.json
-a-----          27/05/2026  01:27 a. m.          5055 README_ESP32_MSSQL.md
-a-----          27/05/2026  12:26 p. m.           2207 servidor_api_esp32.js

PS C:\Users\mr411\OneDrive\Documentos\esp32_mssql_Definitivo-main> node servidor_api_esp32.js
Servidor API corriendo en http://0.0.0.0:3000
Esperando conexiones de ESP32...
```

```
Windows PowerShell  X  Windows PowerShell  X  +  v

Configuración IP de Windows

Adaptador de Ethernet vEthernet (WSL (Hyper-V firewall)):

Sufijo DNS específico para la conexión. . . :
Vínculo: dirección IPv6 local. . . . : fe80::2521:967b:9542:9767%41
Dirección IPv4. . . . . : 172.22.128.1
Máscara de subred . . . . . : 255.255.240.0
Puerta de enlace predeterminada . . . . . :

Adaptador de LAN inalámbrica Conexión de área local* 1:

Estado de los medios. . . . . : medios desconectados
Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Conexión de área local* 2:

Estado de los medios. . . . . : medios desconectados
Sufijo DNS específico para la conexión. . . :

Adaptador de LAN inalámbrica Wi-Fi:

Sufijo DNS específico para la conexión. . . :
Vínculo: dirección IPv6 local. . . . : fe80::46f0:ed2e:dfd4:f2b9%14
Dirección IPv4. . . . . : 192.168.0.103
Máscara de subred . . . . . : 255.255.255.0
Puerta de enlace predeterminada . . . . . : 192.168.0.1
PS C:\Users\mr411> S
```

Ahora abrimos otra pestaña en la cual utilizaremos el comando “ipconfig” para que nos brinde los datos necesarios, pues ya que ocuparemos la dirección IPv4 del Wi-Fi

Capturas de Visual Studio:

Abriremos el documento de servidor_api_esp32.js el cual es el siguiente código:

```
// Servidor REST intermediario para ESP32 -> MSSQL
// npm install express sql mssql cors body-parser

const express = require('express');
const sql = require('mssql');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

// Configuración de MSSQL
const config = {
  server: '192.168.0.103',
  port: 1433,
  user: 'sa',
  password: 'C0NTR453N1!4',
  database: 'baseDePruebas',
  authentication: {
    type: 'default',
    options: {
      userName: 'sa',
      password: 'C0NTR453N1!4'
    }
  },
  options: {
```

```
    encrypt: false,
    trustServerCertificate: true
  }
};

// Ruta para hacer INSERT
app.post('/api/insert', async (req, res) => {
  try {
    // Obtener datos del cuerpo de la petición
    const { nombre, valor, temperatura } = req.body;

    // Conectar a la BD
    let pool = await sql.connect(config);

    // Ejecutar INSERT
    let result = await pool.request()
      .input('nombre', sql.VarChar(50), nombre || 'ESP32')
      .input('valor', sql.Int, valor || Math.floor(Math.random() * 100))
      .input('temperatura', sql.Float, temperatura || 25.5)
      .query('INSERT INTO test (nombre, valor, temperatura) VALUES (@nombre, @valor, @temperatura)');

    // Cerrar conexión
    await pool.close();

    // Respuesta exitosa
    res.json({
      success: true,
```

```
        message: 'Datos insertados correctamente',
        rowsAffected: result.rowsAffected[0]
    });

    } catch (err) {
        console.error('Error:', err);
        res.status(500).json({
            success: false,
            message: 'Error al insertar datos',
            error: err.message
        });
    }
});
```

// Ruta de prueba de conexión

```
app.get('/api/test', async (req, res) => {
    try {
        let pool = await sql.connect(config);
        await pool.close();

        res.json({
            success: true,
            message: 'Conexión a MSSQL exitosa'
        });
    } catch (err) {
        res.status(500).json({
            success: false,
            message: 'Error de conexión a MSSQL',
```

```

        error: err.message
      });
    }
  });

// Iniciar servidor
const PORT = 3000;
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Servidor API corriendo en http://0.0.0.0:\${PORT}`);
  console.log('Esperando conexiones de ESP32...');
});

```

Las partes mas importantes de este codigo o lo que se debe de cambiar para que funcione, es lo siguiente:

```

const config = {
  server: '192.168.0.103',
  port: 1433,
  user: 'sa',
  password: 'C0NTR453N1!4',
  database: 'baseDePruebas',
  authentication: {
    type: 'default',
    options: {
      userName: 'sa',
      password: 'C0NTR453N1!4'
    }
  }
}

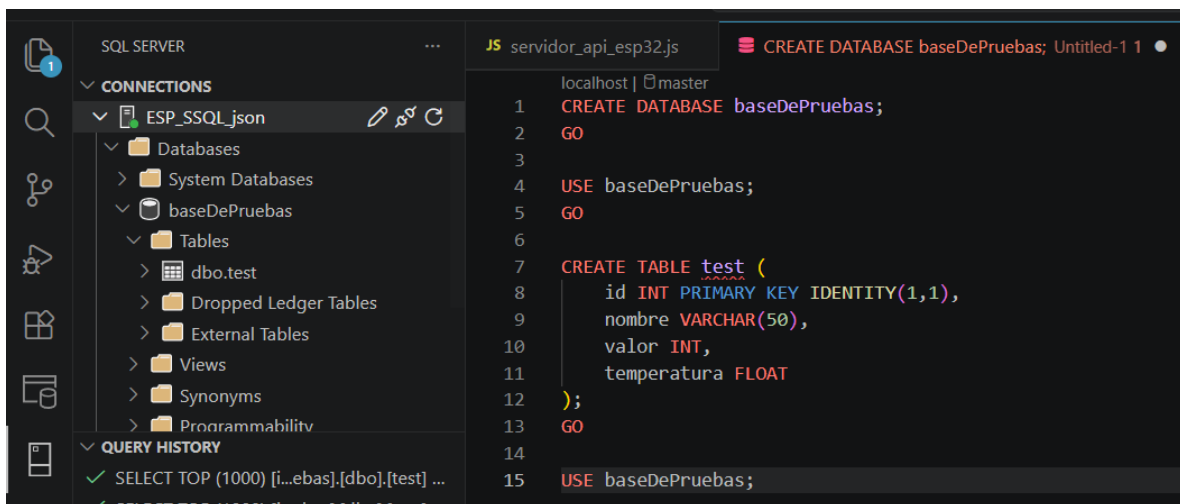
```

En esta parte cambiaremos la parte del server, por la dirección que antes habíamos sacado del PowerShell, en el puerto le pondremos el mismo que pusimos al crear el

contenedor en docker, lo mismo con user y password pondremos los mismos que pusimos al crear el contenedor y por ultimo en el database pondemos el nombre de nuestra base de datos.

```
// Ejecutar INSERT
let result = await pool.request()
  .input('nombre', sql.VarChar(50), nombre || 'ESP32')
  .input('valor', sql.Int, valor || Math.floor(Math.random() * 100))
  .input('temperatura', sql.Float, temperatura || 25.5)
  .query('INSERT INTO test (nombre, valor, temperatura) VALUES (@nombre, @valor, @temperatura)');
```

Estas lineas tambien tenemos que modificarlas de acuerdo a los campos que tengamos en nuestra tabla, y ademas de cambiar el nombre de la tabla, aqui se llama “test” pero debemos de cambiarla de acuerdo como se llame nuestra tabla.



Ahora crearemos una conexión de sql server en visual studio code, para a continuación crear la base de datos con su tabla y sus campos

Código de Arduino IDE

```
// Código para ESP32 S3 - Conexión a MSSQL vía servidor REST

// Librerías necesarias:

// - WiFi (incluida en Arduino IDE para ESP32)
// - HTTPClient (incluida en Arduino IDE para ESP32)

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// ===== CONFIGURACIÓN =====

const char* ssid = "35P32_T35T5"; //conexion a internet
const char* password = "9d81kusrtj"; // conexion a internet
const char* apiUrl = "http://192.168.0.103:3000/api";
const char* testUrl = "http://192.168.0.103:3000/api/test";
const char* insertUrl = "http://192.168.0.103:3000/api/insert";

// Pines de trabajo del ESP32 S3 |Ajusta estos valores según tu proyecto

// Pines de trabajo del ESP32 S3 |Ajusta estos valores según tu proyecto

// Variables de control
unsigned long lastBlink = 0;
bool blinkState = false;
const int BLINK_INTERVAL = 500; // 500ms para parpadeo
```

```

// ===== FUNCIONES LED INTEGRADO=====

void setupLED() {
#define LED_PIN 48

    // Apagar todos los LEDs inicialmente

    digitalWrite(LED_PIN, LOW); // Turn the RGB LED off
}

void setLEDColor(String color) {
    if (color == "off") {
        rgbLedWrite(LED_PIN, 0, 0, 0);
    }
    else if (color == "red") {
        rgbLedWrite(LED_PIN, RGB_BRIGHTNESS, 0, 0); // Red
    }
    else if (color == "green") {
        rgbLedWrite(LED_PIN, 0, RGB_BRIGHTNESS, 0); //Green
    }
    else if (color == "blue") {
        rgbLedWrite(LED_PIN, 0, 0, RGB_BRIGHTNESS); //Blue
    }
}

void blinkLED(String color) {
    unsigned long currentTime = millis();

    if (currentTime - lastBlink >= BLINK_INTERVAL) {
        lastBlink = currentTime;
    }
}

```

```

    blinkState = !blinkState;

    if (blinkState) {
        setLEDColour(color);
    } else {
        setLEDColour("off");
    }
}
}

// ===== FUNCIONES WiFi =====

void connectToWiFi() {
    Serial.println("\n\nInicializando conexión WiFi...");
    Serial.print("Conectando a: ");
    Serial.println(ssid);

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);

    int attemp = 0;
    while (WiFi.status() != WL_CONNECTED && attemp < 20) {
        delay(500);
        Serial.print(".");
        attemp++;

        // Parpadeo azul mientras se conecta
        blinkLED("blue");
    }
}

```

```

}

if (WiFi.status() == WL_CONNECTED) {
    setLEDColour("blue");

    Serial.println("\n✓ Conectado a WiFi");

    Serial.print("IP: ");

    Serial.println(WiFi.localIP());
} else {
    setLEDColour("red");

    Serial.println("\nX Error: No se pudo conectar a WiFi");
}
}

// ===== FUNCIONES HTTP =====

bool testDatabaseConnection() {
    Serial.println("\nProbando conexión a MSSQL...");

    HTTPClient http;
    http.begin(testUrl);
    http.addHeader("Content-Type", "application/json");

    int httpCode = http.GET();

    if (httpCode == 200) {
        String response = http.getString();

        Serial.println("✓ Conexión a BD exitosa");

        Serial.println(response);
    }
}

```

```

        http.end();

        return true;
    } else {

        Serial.print("X Error conectando a BD: ");

        Serial.println(httpCode);

        http.end();

        return false;
    }
}

bool insertDataToDatabase(const char* nombre, int valor, float temperatura)
{
    Serial.println("\nEnviando INSERT a MSSQL...");

    HTTPClient http;

    http.begin(insertUrl);

    http.addHeader("Content-Type", "application/json");

    // Crear JSON con los datos
    DynamicJsonDocument doc(200);

    doc["nombre"] = nombre;

    doc["valor"] = valor;

    doc["temperatura"] = temperatura;

    String jsonString;

    serializeJson(doc, jsonString);

    Serial.print("Datos enviados: ");

```

```
Serial.println(jsonString);

int httpCode = http.POST(jsonString);

if (httpCode == 200) {
    String response = http.getString();
    Serial.println("✓ INSERT exitoso");
    Serial.println(response);
    http.end();
    return true;
} else {
    Serial.print("X Error en INSERT: ");
    Serial.println(httpCode);
    String response = http.getString();
    Serial.println(response);
    http.end();
    return false;
}
}

// ===== SETUP =====

void setup() {

    Serial.begin(115200);
    delay(1000);

    Serial.println("\n\n=== ESP32 S3 MSSQL DEBUG ===");
}
```

```
setupLED();

// Conectar a WiFi
connectToWiFi();

if (WiFi.status() != WL_CONNECTED) {
    setLEDColour("red");
    Serial.println("No se pudo conectar a WiFi. Reiniciando en 5 segundos...");
    delay(5000);
    ESP.restart();
}

// Parpadeo azul mientras se prueba conexión a BD
Serial.println("\nProbando conexión a base de datos...");
bool connectionOk = false;

for (int i = 0; i < 10; i++) {
    blinkLED("blue");

    if (testDatabaseConnection()) {
        connectionOk = true;
        break;
    }
}
```



```

    }

    delay(500);
}

if (!connectionOk) {
    setLEDColour("red");

    Serial.println("No se pudo conectar a la BD. Verifique la
configuración.");

    delay(5000);

    ESP.restart();
}

// Hacer un INSERT de prueba
Serial.println("\nRealizando INSERT de prueba...");

if (insertDataToDatabase("ESP32_S3_Test", 42, 23.5)) {
    // INSERT exitoso - LED verde

    setLEDColour("green");

    Serial.println("\n✓✓✓ TODO CORRECTO - LED VERDE ✓✓✓");
} else {
    // INSERT fallido - LED rojo

    setLEDColour("red");

    Serial.println("\nXXX ERROR EN INSERT - LED ROJO XXX");
}
}

// ===== LOOP =====

```

```
void loop() {  
  
    // Mantener el LED en el estado actual  
    // El color ya está seteado (verde o rojo)  
  
    // Cada 30 segundos, hacer otro INSERT para demostración  
    static unsigned long lastInsert = 0;  
    if (millis() - lastInsert > 30000) {  
        lastInsert = millis();  
  
        Serial.println("\n\n=== Enviando INSERT periódico ===");  
  
        int valorRandom = random(0, 100);  
        float temperaturaRandom = 20.0 + (random(0, 50) / 10.0);  
  
        if (insertDataToDatabase("ESP32_Periodic", valorRandom,  
temperaturaRandom)) {  
            setLEDColour("green");  
            Serial.println("✓ INSERT periódico exitoso");  
  
        } else {  
            setLEDColour("red");  
            Serial.println("X INSERT periódico fallido");  
  
        }  
    }  
}  
  
delay(100);
```

```
setLEDColour("off");  
}
```

```
10 // ===== CONFIGURACIÓN =====  
11 const char* ssid = "35P32_T35T5"; //conexion a internet  
12 const char* password = "9d81kusrtj"; // conexion a internet  
13 const char* apiUrl = "http://192.168.0.103:3000/api";  
14 const char* testUrl = "http://192.168.0.103:3000/api/test";  
15 const char* insertUrl = "http://192.168.0.103:3000/api/insert";  
16
```

Las únicas partes que debemos de modificar de este código sería en esta parte, pues ya que cambiaremos las líneas 11 y 12 de acuerdo con el dispositivo que estemos conectados, como un wifi o un dispositivo, pondremos el nombre del wifi y a continuación la contraseña de este mismo, en las líneas 13, 14 y 15 las dejaremos igual, pero cambiaremos la dirección IP por la misma que antes habíamos sacado del PowerShell.

Capturas de la Practica

```
11 const char* ssid = "INFINITUM31D7"; //conexion a internet
12 const char* password = "nQcvkDR3D4"; // conexion a internet
13 const char* apiUrl = "http://192.168.1.74:3000/api";
14 const char* testUrl = "http://192.168.1.74:3000/api/test";
15 const char* insertUrl = "http://192.168.1.74:3000/api/insert";
16
17 // Pines de trabajo del ESP32 S3 |Ajusta estos valores según tu proyecto
18
19 // Pines de trabajo del ESP32 S3 |Ajusta estos valores según tu proyecto
20
21
```

Salida Monitor Serie X

Mensaje (Intro para mandar el mensaje de 'ESP32 Dev Module' a 'COM3')

Realizando conexión a BD...

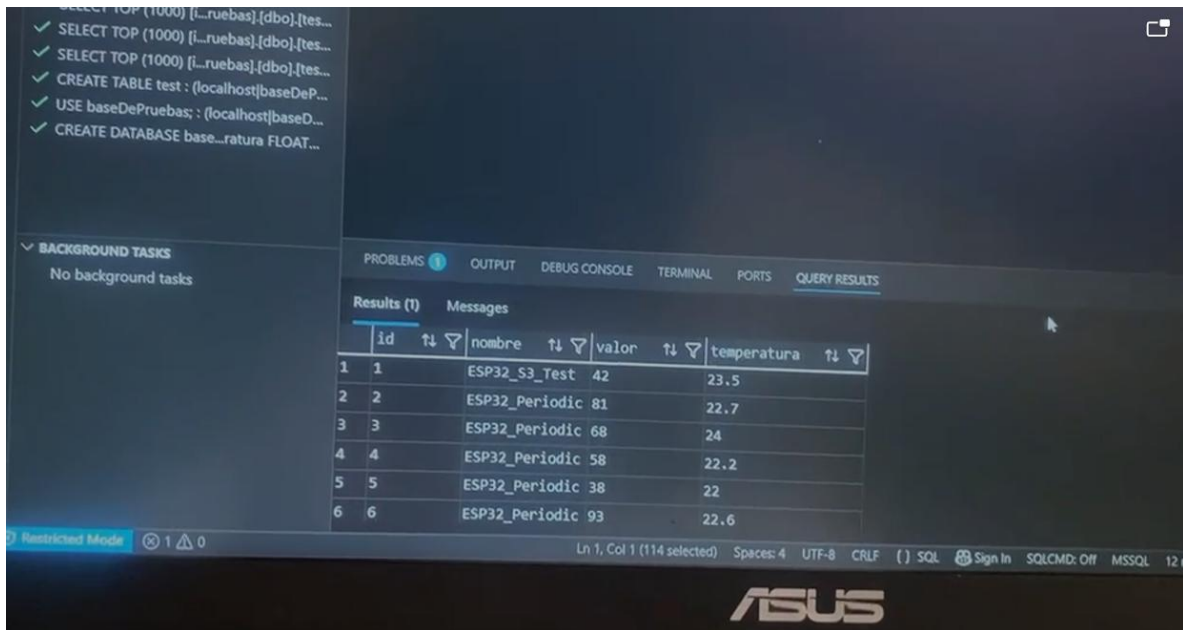
✓ Conexión a BD exitosa
{ "success": true, "message": "Conexión a MSSQL exitosa" }

Realizando INSERT de prueba...

Enviando INSERT a MSSQL...

Datos enviados: { "nombre": "ESP32_S3_Test", "valor": 42, "temperatura": 23.5 }

✓ INSERT exitoso
{ "success": true, "message": "Datos insertados correctamente", "rowsAffected": 1 }



Conclusión

Manuel Humberto Reyes Salas: Este proyecto me permitió comprender cómo se integra el hardware con el software en un sistema IoT real, entendiendo el papel de la ESP32 como dispositivo de captura y envío de datos mediante WiFi hacia un servidor.

María José Guerrero Rios: Aprendí la importancia de utilizar una API como intermediario entre el dispositivo y la base de datos, ya que facilita la comunicación, el procesamiento de información y la seguridad en el manejo de los datos antes de almacenarlos en SQL Server.

Jesús Eli Sánchez Ruvalcaba: Reforcé mis conocimientos sobre el uso de tecnologías como Docker y NodeJS, entendiendo cómo ayudan a crear entornos de desarrollo más organizados, escalables y fáciles de implementar en proyectos reales.

Link de video:

<https://drive.google.com/file/d/17g71p3Lc7OWH3zWW4KekcbRc9dBv-wHD/view?usp=sharing>

Link de repositorio:

https://github.com/MariaJose3535/-WiFi-3_ESP32

